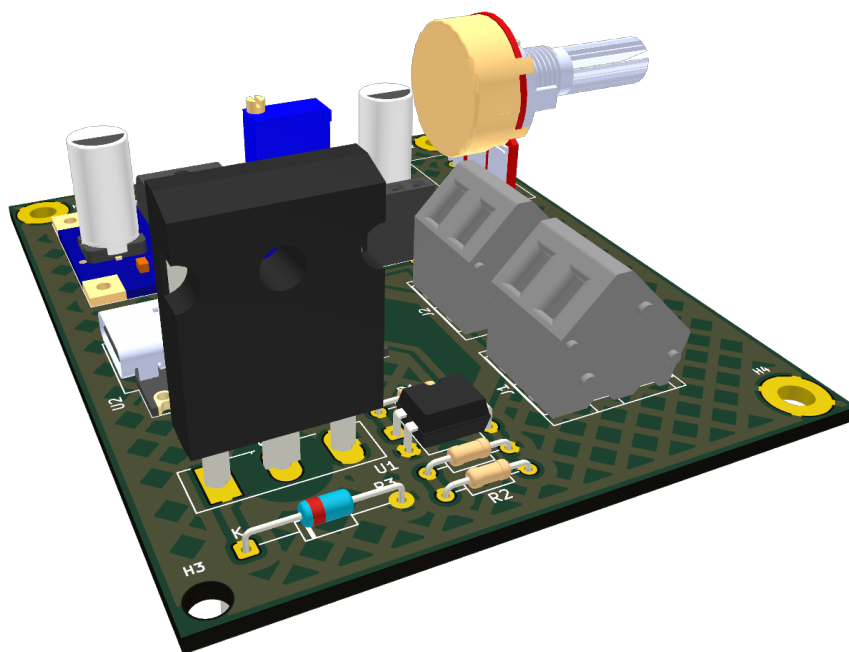




Probador de Bobinas COP

Generador de disparos PWM con dwell controlado para bobinas de encendido
ESP32-C3 · IGBT 600 V · 10–200 Hz · LCD 20x4



designed by Ramiro · Revisión 2 · julio 2026

github.com/ramiroedson72-cloud/probador-bobinas-cop

Índice

1. Descripción general	3
2. Especificaciones	3
3. Firmware y funcionamiento	4
4. La placa (PCB)	5
5. Lista de materiales (BOM)	7
6. Biblioteca de componentes	7
7. Fabricación	8
8. Puesta en marcha (primer arranque)	8
9. Solución de problemas	8
10. Conexión y seguridad	10
11. Carcasa	10
12. Historial del diseño	10
13. Repositorio	10
14. Código fuente comentado (Probador_Bobinas.ino)	12
Anexo A: Esquemático completo — última página	

1. Descripción general

El probador genera pulsos de disparo para bobinas de encendido tipo COP (Coil-On-Plug) simulando el trabajo real del motor: un potenciómetro ajusta la frecuencia de chispa entre 10 y 200 Hz (300–6000 RPM en un motor de 4 cilindros) y el firmware mantiene un **dwell fijo de 3 ms** recalculando el ciclo de trabajo del PWM en cada ajuste. El estado se muestra en una LCD 20x4 I2C (opcional, con autodetección) y por el monitor serial.

La cadena de disparo es: **ESP32-C3** → **optoacoplador PC817** → **IGBT FGH60N60SFD** → **primario de la bobina**. El optoacoplador aísla galvánicamente la lógica de 3.3 V del lado de potencia de 12 V, y el IGBT de 600 V absorbe con margen el flyback de 300–400 V que genera el primario en cada corte. Un zener de 15 V protege el gate y un módulo buck LM2596 genera los 5 V lógicos.

2. Especificaciones

Parámetro	Valor
Alimentación	12 V DC (batería o fuente, picos de 5-10 A)
Frecuencia de disparo	10 - 200 Hz (potenciómetro) = 300 - 6000 RPM en 4 cil.
Dwell	3 ms fijos; sobre ~133 Hz lo limita el tope de duty del 40%
Corriente de primario	~3 A con primario de 3.2 Ω (hasta ~10 A según bobina)
Flyback soportado	300 - 400 V (IGBT de 600 V con margen)
Microcontrolador	ESP32-C3 Super Mini (USB-C)
Display	LCD 20x4 I2C (PCF8574) - opcional, autodetectado en 0x27/0x3F
Placa	61 x 74 x 1.6 mm, 2 capas, THT
Altura máx. componentes	~24 mm (IGBT TO-247)

3. Firmware y funcionamiento

El sketch (Probador_Bobinas.ino, Arduino) usa el periférico LEDC del ESP32-C3 a **14 bits de resolución**, lo que permite generar con precisión todo el rango de 10 a 200 Hz. El lazo principal promedia 16 lecturas del potenciómetro, mapea el valor a frecuencia y solo reconfigura el PWM cuando el cambio es real (≥ 2 Hz) para evitar glitches en el tren de pulsos. El duty se recalcula como *dwell x frecuencia*, con tope de seguridad del 40% — por eso a más de ~133 Hz el dwell efectivo baja gradualmente de 3 ms (protege bobina e IGBT a altas RPM).

3.1 Asignación de pines

Pin ESP32-C3	Función
GPIO3 (ADC)	Cursor del potenciómetro (extremos a 3V3 y GND)
GPIO4	Salida PWM → R1 150 Ω → LED del PC817
GPIO8	SDA de la LCD 20x4 I2C
GPIO9	SCL de la LCD 20x4 I2C
5V	Alimentación desde el LM2596 (vía red de 5 V de la placa)

3.2 Interfaz

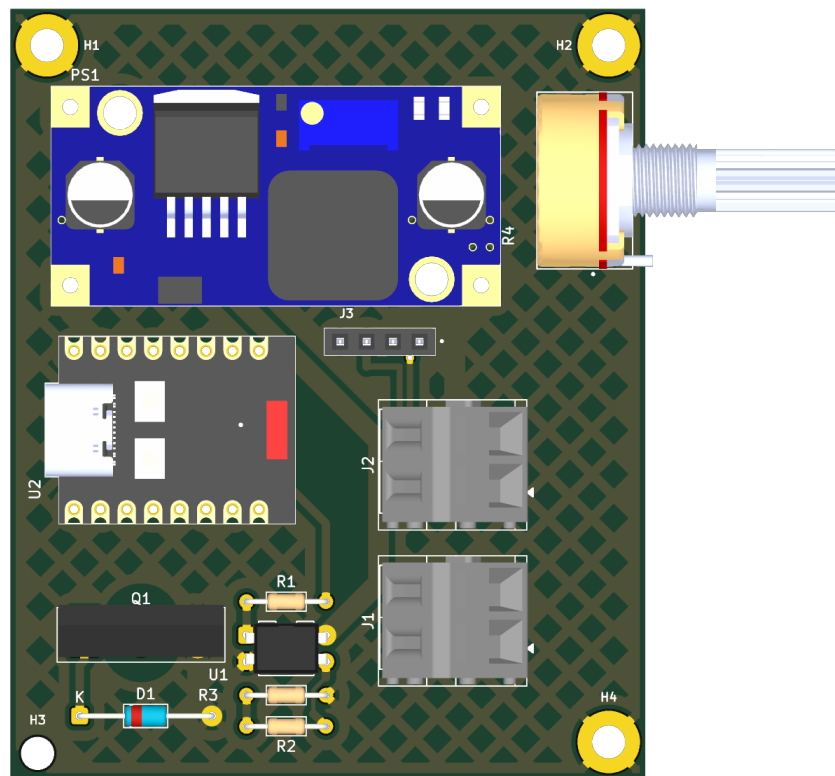
La LCD muestra frecuencia real del LEDC, RPM equivalentes (4 cilindros), dwell y duty. Si no hay LCD conectada el probador funciona igual y reporta por serial a 115200 baudios una línea cada 200 ms con el ADC crudo del potenciómetro (útil para diagnosticarlo: debe recorrer ~0 a ~4095), la frecuencia objetivo, la frecuencia real y el duty aplicado.

3.3 Compilación

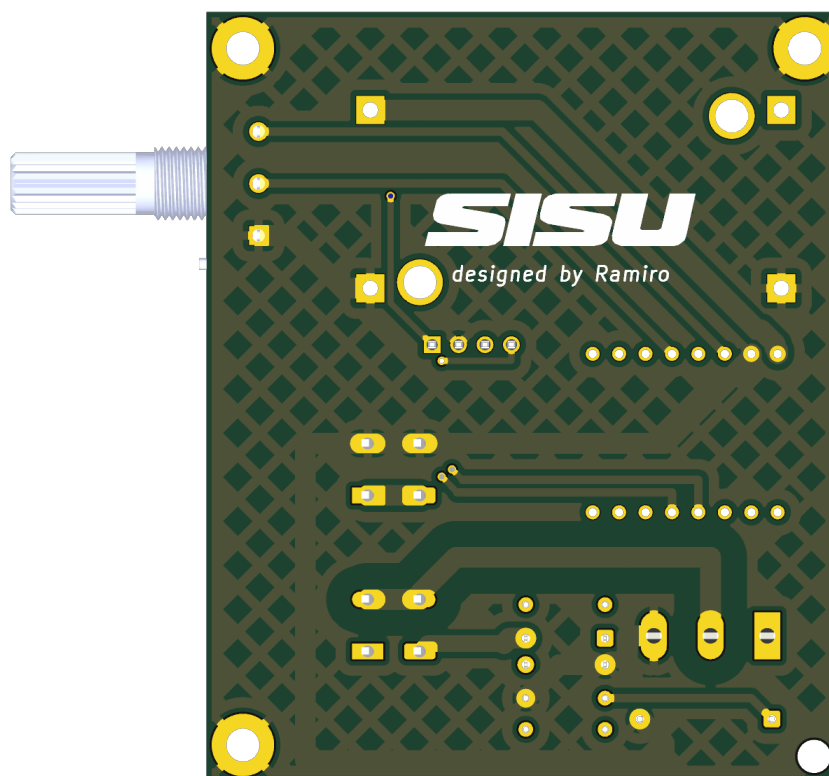
Core ESP32 de Arduino v2.x o v3.x (el sketch detecta la API de LEDC automáticamente). Librería LiquidCrystal I2C (Frank de Brabander). Placa: ESP32C3 Dev Module con **USB CDC On Boot: Enabled**. Si no entra a programación: mantener BOOT, pulsar RESET, soltar BOOT.

4. La placa (PCB)

Diseñada en KiCad 10, 100% THT para armado a mano. Enrutado con Freerouting más retoques manuales bajo reglas por clases de red: la pista del colector del IGBT (flyback de ~400 V) exige **2.5 mm de separación**; la potencia va a 2 mm; las señales a 0.8 mm (aptas para tóner). Ambas caras llevan **plano de tierra cuadrículado** (1 mm / hueco 2 mm a 45°) que drena las señales parásitas, y los taladros M3 de H1, H2 y H4 tienen **anillo a GND** para aterrizar la carcasa metálica con los propios tornillos (H3 es solo mecánico). La cara inferior lleva el logo SISU y la leyenda *designed by Ramiro*. Verificación DRC final: cero violaciones.



Cara superior: LM2596 (PS1), potenciómetro al borde, ESP32-C3 (U2), clemas WAGO (J1/J2), IGBT (Q1) y etapa de disparo.



Cara inferior: pistas, malla GND, logo SISU. La pista ancha diagonal es el colector (alta tensión).

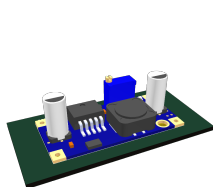
5. Lista de materiales (BOM)

Ref.	Componente	Cant.
D1	D_Zener 15V	1
J1,J2	WAGO_236-402	2
J3	PPTC041LFBN-RC	1
PS1	MC00100	1
Q1	FGH60N60SFD	1
R1	R_150	1
R2	R_100	1
R3	R_560	1
R4	WH148-1A-2-18T-L20	1
U1	PC817	1
U2	ESP32-C3_SUPERMINI_TH	1
-	LCD 20x4 I2C (PCF8574) - externo vía J3	1
-	Fusible rápido 5 A con portafusible (externo)	1
H1-H4	Tornillos M3 (latón o acero) - H1/H2/H4 aterrizan la carcasa	4

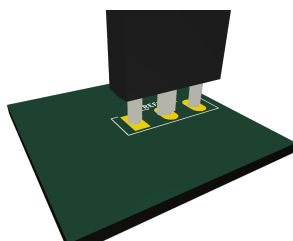
Valores: R1 150 Ω (serie del LED del opto), R2 100 Ω (serie de gate), R3 560 Ω (pull-down de gate), R4 potenciómetro WH148 10 k Ω , D1 zener 15 V gate-emisor.

6. Biblioteca de componentes

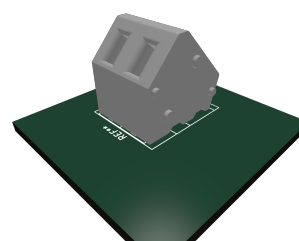
Las partes no estándar se integraron a una librería KiCad propia (símbolo + footprint + modelo 3D verificado por render). La librería viaja con el repositorio (libreria_kicad_SnapEDA) para que el proyecto abra completo en cualquier PC.



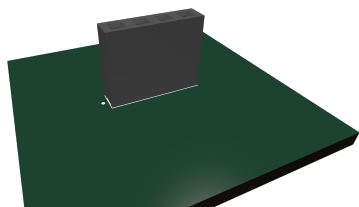
Módulo LM2596



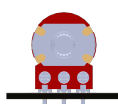
IGBT TO-247



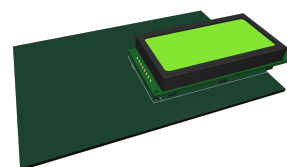
Clema WAGO 236-402



Header hembra 1x4



Potenciómetro WH148



LCD 20x4 (para carcasa)

7. Fabricación

7.1 Profesional (JLCPCB / PCBWay)

Subir **PARA_FABRICAR/Probador_Bobinas_GERBERS.zip** tal cual. Opciones: 2 capas, 1.6 mm, cobre 1 oz, resto por defecto. El PCBA no aplica: el ESP32 SuperMini y el LM2596 son módulos de aftermarket que se sueldan a mano.

7.2 Casera (transferencia de tóner + máscara UV)

PDFs a escala 1:1 en **PCB bobina Imprimir** (láser al 100%; el contorno debe medir 61 x 74 mm):

Archivo	Uso
planchar_COBRE_ABAJO_no_espejar	Cobre inferior - se imprime tal cual
planchar_COBRE_ARRIBA_ya_espejado	Cobre superior - ya viene espejado
mascaraUV_ABAJO / ARRIBA	Acetatos de máscara UV (pads en negro; 2 copias por cara encimadas)
serigrafia_ARRIBA / ABAJO	Serigrafía de componentes (arriba) y logo (abajo)

Las 7 vías se hacen con alambre pasante soldado por ambos lados. Las tres rutas de alta corriente están marcadas con línea punteada en la capa Cmts.User (retorno GND, colector 300 V y VCC de bobina): en la versión casera se refuerzan estañando un alambre de cobre desnudo (~18-20 AWG) sobre la pista. En la placa de fábrica no es necesario.

8. Puesta en marcha (primer arranque)

Seguir el orden: cada paso protege al siguiente.

1. Inspección visual y de continuidad: revisar con multímetro que no haya puentes entre pistas vecinas, en especial el corredor entre J3 y la clema J2 (pistas de 0.6 mm) si la placa es casera.
2. **SIN el ESP32 insertado y sin bobina:** conectar 12 V a J2 (con fusible de 5 A) y ajustar el trimmer del LM2596 hasta medir **5.0 V** entre el pin 1 de J3 y GND. Este paso evita quemar el ESP32 si el módulo viene ajustado alto de fábrica.
3. Cortar alimentación, insertar el ESP32 (USB hacia el borde izquierdo) y reconectar. Abrir el monitor serial (115200): girar el potenciómetro y verificar que el ADC recorra ~0 a ~4095 y la frecuencia real siga a la objetivo.
4. Conectar la bobina COP en J1 y una bujía en su torre, con el **cuerpo de la bujía firmemente a GND de potencia** (sin este retorno no hay chispa y el flyback sube de más).
5. Arrancar en 10 Hz (potenciómetro al mínimo) y subir gradualmente. La chispa debe ser azul y constante en todo el rango.

9. Solución de problemas

Síntoma	Causa probable	Qué hacer
No hay chispa	Bujía sin retorno a GND / bobina mal conectada / fusible abierto	Verificar cuerpo de bujía a GND de potencia, continuidad de J1 y fusible
Chispa débil o intermitente	Dwell insuficiente a alta frecuencia / 12 V caídos	Probar a baja frecuencia; alimentar con batería o fuente de 5-10 A
El ESP32 se reinicia al chispear	EMI del secundario / cables de AT junto a la lógica	Alejar bobina y cables de bujía de la placa; verificar malla GND y carcasa aterrizada
LCD no enciende o basura	Dirección I2C distinta / contraste / cables SDA-SCL cruzados	El firmware busca 0x27 y 0x3F; ajustar trimmer del backpack; revisar J3

Síntoma	Causa probable	Qué hacer
IGBT muy caliente	Sesión larga a alta frecuencia sin disipador	Atornillar disipador al TO-247 (hay espacio junto a Q1); pausar entre pruebas
ADC no llega a 0 o a 4095	Potenciómetro mal cableado o dañado	Ver monitor serial; revisar soldaduras de R4 (extremos a 3V3/GND, cursor a GPIO3)

10. Conexión y seguridad

Punto	Conexión
Clema J2	Entrada 12 V (con fusible externo de 5 A rápido)
Clema J1	Primario de la bobina COP
Header J3	LCD 20x4 I2C: 5V - GND - SDA (GPIO8) - SCL (GPIO9)
Bujía	El cuerpo (rosca) va a GND de potencia
Potenciómetro	Frecuencia de disparo (10-200 Hz)
USB-C del ESP32	Reprogramación y monitor serial

Seguridad: el secundario de una bobina COP supera los 30 kV — nunca tocar la bujía ni su torre con el equipo energizado. Probar siempre con bujía o chispómetro conectado. No agregar diodo flyback ni TVS al primario (mata la chispa). Personas con marcapasos deben mantener distancia. Trabajar en área ventilada y sin combustibles cerca.

11. Carcasa

Placa de 61 x 74 mm; taladros M3 en H1 (93.5, 41.5), H2 (147.5, 41.5), H3 (92.6, 109.6) y H4 (147.5, 108.5) en coordenadas del proyecto. Altura interior mínima 35 mm. El eje del potenciómetro sale ~20 mm por el borde derecho (atraviesa el panel); las clemas miran al mismo lado; el USB-C queda a la izquierda (prever ranura). Si la carcasa es metálica, los tornillos de H1/H2/H4 la aterrizan al plano GND (usar rondana dentada en al menos uno). El LCD 20x4 (98 x 60 mm, ventana 76 x 25 mm, taladros 93 x 55 mm) requiere ~25 mm de profundidad con su backpack I2C. Modelos 3D en el repositorio: Probador_Bobinas_3D.step y LCD_20x4_NHD-0420H1Z.step.

12. Historial del diseño

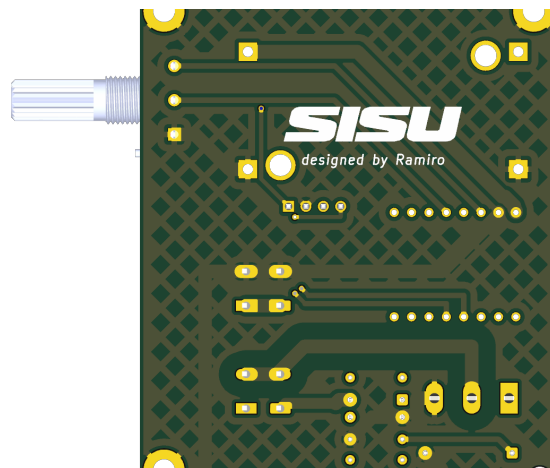
Versión	Descripción	Estado
v1 - protoboard	IRLZ44N (MOSFET 55 V): el flyback se disipaba en avalancha dentro del transistor. A alta frecuencia la disipación lo quemó; se limitó el firmware a 60 Hz y dwell 2 ms.	Superada
v2 - PCB (esta)	IGBT FGH60N60SFD de 600 V: absorbe el flyback con margen. Rango completo de 10-200 Hz y dwell de 3 ms restaurados. PCB con reglas de alta tensión, malla GND y carcasa aterrizable.	Actual

13. Repositorio

Proyecto completo (público): github.com/ramiroedson72-cloud/probador-bobinas-cop — firmware Arduino, proyecto KiCad, gerbers, PDFs de fabricación casera, librería de componentes autocontenida, STEP 3D, esquemático PDF, BOM y esta documentación.



Escanea para abrir el repositorio en GitHub



Detalle de la cara inferior: logo SISU y designed by Ramiro en serigrafía.

14. Código fuente comentado (Probador_Bobinas.ino)

El firmware cabe en un solo sketch de ~165 líneas. Su estructura:

Bloque	Qué hace
Configuración	Pines (pot GPIO3, PWM GPIO4, I2C 8/9) y constantes: FREQ 10-200 Hz, DWELL 3 ms, tope de duty 40%, LEDC a 14 bits (necesarios para que las frecuencias bajas no fallen en silencio).
detectarLcd()	Busca el backpack PCF8574 en 0x27 y 0x3F; si no aparece, el equipo opera solo con el monitor serial.
aplicarPwm() / freqReal()	Capa de compatibilidad entre el core ESP32 v2.x (API de canales) y v3.x (API por pin), con verificación de que el LEDC aceptó la frecuencia.
dutyParaDwell()	Convierte el dwell fijo en duty proporcional a la frecuencia ($duty = dwell \times f$), recortado al 40% de seguridad.
setup()	Serial, I2C + LCD, ADC a 12 bits y arranque del PWM en 10 Hz; si el LEDC falla, se detiene con mensaje de error.
loop()	Promedia 16 lecturas del pot, mapea a frecuencia y solo reconfigura el LEDC con cambios reales (≥ 2 Hz) para no glitchear el tren de pulsos; refresca LCD/serial cada 200 ms con frecuencia real, RPM, dwell y duty.

14.1 Listado completo

```
/*
 * Probador de bobinas de encendido COP
 * ESP32-C3 Super Mini + PC817 + IGBT FGH60N60SFD (PCB v2)
 *
 * El potenciómetro controla la frecuencia de disparo (10-200 Hz).
 * El dwell (tiempo de carga de la bobina) se mantiene fijo en 3 ms
 * recalculando el duty en cada cambio, con tope de seguridad del 40%
 * (a mas de ~133 Hz el tope reduce el dwell gradualmente).
 *
 * Conexiones (segun PCB v2):
 * GPIO3 <- cursor del potenciómetro WH148 (extremos a 3V3 y GND)
 * GPIO4 -> R1 150R -> pin 1 PC817 (pin 2 a GND logico)
 * GPIO8 -> SDA de la LCD 20x4 (backpack PCF8574, alimentada a 5V)
 * GPIO9 -> SCL de la LCD
 *
 * Compatible con el core ESP32 de Arduino v2.x y v3.x.
 */

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ----- Configuración -----
const int PIN_POT = 3; // entrada ADC del potenciómetro
const int PIN_PWM = 4; // salida hacia el opto/MOSFET
const int PIN_SDA = 8;
const int PIN_SCL = 9;

const int FREQ_MIN = 10; // Hz (= 300 RPM en motor 4 cil.)
// Con el IGBT FGH60N60SFD (600 V) el flyback de la bobina ya no se disipa
// en avalancha: el rango completo es seguro. (Con el IRLZ44N del prototipo
// v1 habia que limitar a 60 Hz porque el MOSFET clampeaba a 55 V y ardia.)
const int FREQ_MAX = 200; // Hz (= 6000 RPM en motor 4 cil.)
// 3 ms: dwell típico de bobina COP (primario 3.2 ohms -> ~3 A de pico).
const float DWELL_MS = 3.0; // tiempo de carga de la bobina
const float DUTY_MAX = 0.40; // tope de seguridad de duty (40%)

// 14 bits: con el reloj de 80 MHz el LEDC acepta de 4.8 Hz a 4.8 kHz,
// así todo el rango 10-200 Hz es válido. Con 12 bits el mínimo era
// ~19 Hz y las frecuencias bajas fallaban en silencio.
const int RES_BITS = 14;
const uint32_t DUTY_FULL = (1UL << RES_BITS) - 1;

#if !defined(ESP_ARDUINO_VERSION_MAJOR) || ESP_ARDUINO_VERSION_MAJOR < 3
#define CORE_V2 // API antigua de LEDC (canales)
const int PWM_CHANNEL = 0;
#endif

LiquidCrystal_I2C *lcd = nullptr;
bool hayLcd = false;

int freqActual = 0; // frecuencia aplicada al LEDC
uint32_t lastLcd = 0;

// Busca la LCD en las dos direcciones típicas del backpack PCF8574
```

```

uint8_t detectarLcd() {
    const uint8_t dirs[] = {0x27, 0x3F};
    for (uint8_t d : dirs) {
        Wire.beginTransmission(d);

        if (Wire.endTransmission() == 0) return d;
    }
    return 0;
}

// Devuelve la frecuencia que el LEDC esta generando de verdad
uint32_t freqReal() {
#ifdef CORE_V2
    return ledcReadFreq(PWM_CHANNEL);
#else
    return ledcReadFreq(PIN_PWM);
#endif
}

// Reconfigura el LEDC. Devuelve false si el driver rechazo la frecuencia.
bool aplicarPwm(int freq, uint32_t duty) {
#ifdef CORE_V2
    bool ok = ledcChangeFrequency(PWM_CHANNEL, freq, RES_BITS) != 0;
    ledcWrite(PWM_CHANNEL, duty);
#else
    bool ok = ledcChangeFrequency(PIN_PWM, freq, RES_BITS) != 0;
    ledcWrite(PIN_PWM, duty);
#endif
    return ok;
}

uint32_t dutyParaDwell(int freq) {
    float dutyFrac = DWELL_MS * freq / 1000.0f;
    if (dutyFrac > DUTY_MAX) dutyFrac = DUTY_MAX;
    return dutyFrac * DUTY_FULL;
}

void setup() {
    Serial.begin(115200);

    Wire.begin(PIN_SDA, PIN_SCL);
    uint8_t dir = detectarLcd();
    if (dir != 0) {
        lcd = new LiquidCrystal_I2C(dir, 20, 4);
        lcd->init();
        lcd->backlight();
        lcd->setCursor(0, 0);
        lcd->print("Probador de bobinas");
        hayLcd = true;
        Serial.printf("LCD encontrada en 0x%02X\n", dir);
    } else {
        Serial.println("LCD no detectada, siga solo con Serial");
    }

    analogReadResolution(12);

#ifdef CORE_V2
    ledcSetup(PWM_CHANNEL, FREQ_MIN, RES_BITS);
    ledcAttachPin(PIN_PWM, PWM_CHANNEL);
    bool pwmOk = true;
#else
    bool pwmOk = ledcAttach(PIN_PWM, FREQ_MIN, RES_BITS);
#endif

    if (!pwmOk) {
        Serial.println("ERROR: fallo la configuracion del PWM (LEDC);");
        if (hayLcd) { lcd->setCursor(0, 1); lcd->print("ERROR config PWM!"); }
        while (true) delay(1000); // no tiene caso seguir
    }

    freqActual = FREQ_MIN;
    aplicarPwm(FREQ_MIN, dutyParaDwell(FREQ_MIN));
}

void loop() {
    // Promedio de 16 lecturas para estabilizar el pot
    long suma = 0;
    for (int i = 0; i < 16; i++) suma += analogRead(PIN_POT);
    int adc = suma / 16;

    int freq = map(adc, 0, 4095, FREQ_MIN, FREQ_MAX);

    // Solo reconfigurar el LEDC cuando el cambio es real (>= 2 Hz).
    // Reconfigurar en cada vuelta del loop reinicia el temporizador
    // antes de completar un periodo y el PWM sale glitcheado.
    if (abs(freq - freqActual) >= 2) {
        freqActual = freq;
        if (!aplicarPwm(freqActual, dutyParaDwell(freqActual))) {
            Serial.printf("ERROR: LEDC rechazo %d Hz\n", freqActual);
        }
    }

    // Refresco de pantalla y serial cada 200 ms
    if (millis() - lastLcd >= 200) {

```

```

lastLcd = millis();
int rpm = freqActual * 30; // motor 4 tiempos, 4 cilindros
int dutyPct = int(100.0f * dutyParaDwell(freqActual) / DUTY_FULL);

if (hayLcd) {
    char linea[21];
    snprintf(linea, sizeof(linea), "Frec: %3d Hz      ", (int)freqReal());
    lcd->setCursor(0, 1); lcd->print(linea);
    snprintf(linea, sizeof(linea), "RPM (4cil): %5d    ", rpm);
    lcd->setCursor(0, 2); lcd->print(linea);
    snprintf(linea, sizeof(linea), "Dwell: %.1fms D: %2d%%    ", DWELL_MS, dutyPct);
    lcd->setCursor(0, 3); lcd->print(linea);
}
// adc crudo para diagnosticar el pot: debe ir de ~0 a ~4095 al girarlo
Serial.printf("adc=%4d f_obj=%3d Hz f_real=%3u Hz duty=%d%%\n",
              adc, freqActual, freqReal(), dutyPct);
}
}

```

